

ST2 Assignment 2- Group Project

Student ID	u3273121, u3290462
Tutorial details	Wednesday, 11:30 am

Project overview:

DSA Explorer is an interactive Python and Pygame-based educational application designed to help users understand core Data Structures and Algorithms (DSA) through real-time visualisation, animation and user interaction. The system combines multiple DSA concepts into one integrated platform with a modular menu-driven interface.

The project was developed across three implementation phases and includes both algorithm logic and graphical visualisation modules. Each module allows users to interact with the data structure or algorithm using keyboard controls while observing how the internal structure changes dynamically.

The application focuses on improving conceptual understanding of algorithms through visual feedback. Instead of only displaying textual outputs, DSA Explorer visually demonstrates operations such as insertion, deletion, traversal, sorting, path calculation, heap ordering and graph relationships.

The project was implemented using:

- Python
- Pygame
- Object-Oriented Programming Principles
- Modular File Architecture
- Automated Unit Testing

The final system includes the following modules:

- Stack Visualiser
- Queue Visualiser

- Linked List Visualiser
- Binary Search Tree (BST) Visualiser
- Heap Visualiser
- Graph Visualiser
- Dynamic Programming (DP) Visualiser
- Sorting Visualiser

All modules are connected through a unified main menu interface for seamless navigation.

1. Phase 1 Implementation

1.1 Stack Visualiser

The Stack Visualiser demonstrates the Last-In-First-Out (LIFO) principle.

Features

- Push operation
- Pop operation
- Dynamic stack rendering
- Keyboard interaction
- Real-time visual updates

Algorithms Used

- Stack insertion
- Stack deletion

User Interaction

- SPACE key pushes data onto the stack.
- BACKSPACE removes the top item.
- ESC returns to the main menu.

Visualisation

Animated blocks are displayed vertically to represent stack order. The latest inserted element appears at the top.

Learning Outcome

This module demonstrates how stack operations manipulate memory in sequential order.

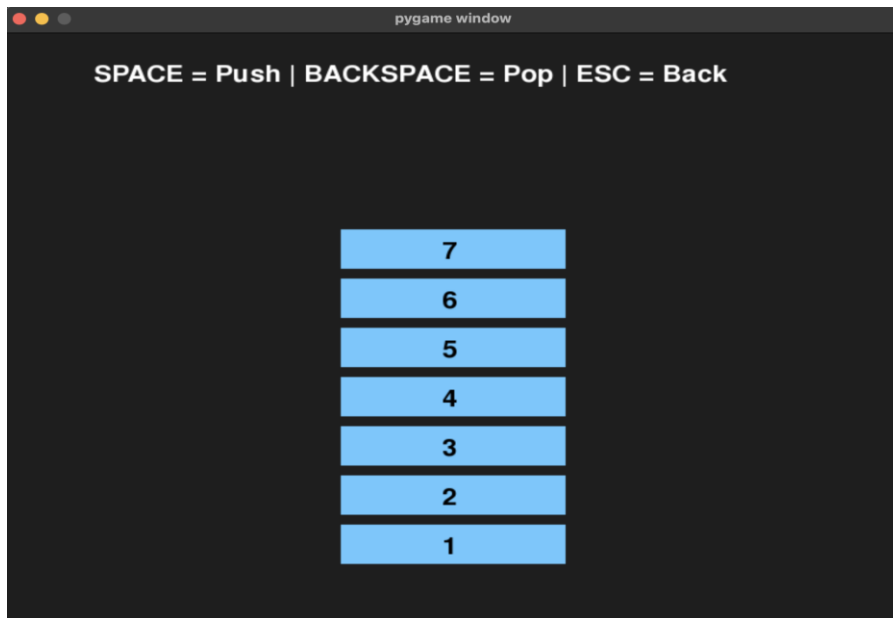


Fig 1: stack visualizer

1.2 Queue Visualiser

The Queue Visualiser demonstrates the First-In-First-Out (FIFO) principle.

Features

- Enqueue operation
- Dequeue operation
- Dynamic queue rendering

Algorithms Used

- Queue insertion
- Queue removal

User Interaction

- SPACE begins input mode.
- BACKSPACE dequeues the front item.
- ESC exits the module.

Visualisation

Queue items are displayed horizontally in insertion order.

Improvements

The queue system was improved from automatic number generation to user-controlled queue insertion to enhance interactivity and reduce hardcoded behaviour.

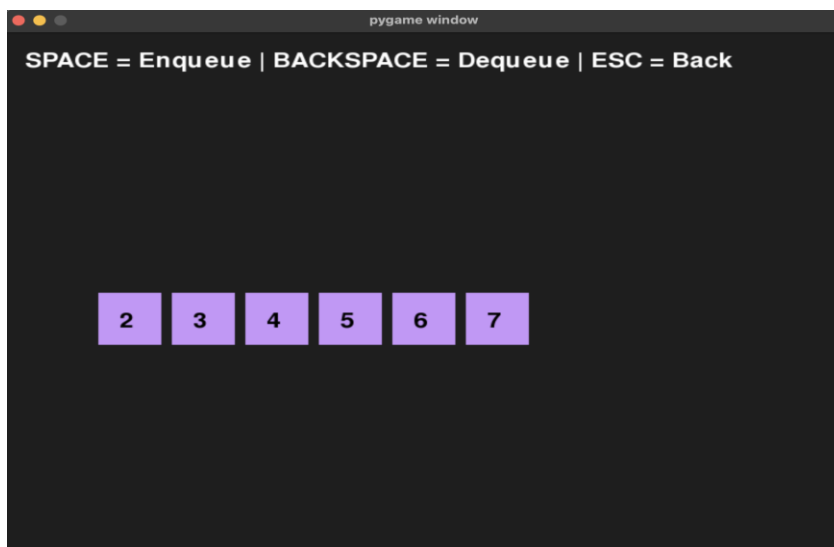
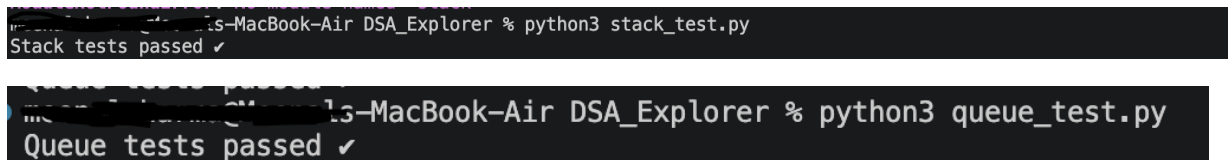


Fig 2: Queue visualizer



2. Phase 2 Implementation

2.1 Linked List Visualiser

The Linked List Visualiser demonstrates node-based dynamic memory structures.

Features

- Insert node
- Delete node
- Reverse linked list
- Pointer visualisation
- Interactive controls

Algorithms Used

- Node insertion
- Node deletion
- List traversal
- Reverse traversal

Visualisation

Nodes are connected using directional pointers to visually represent links between memory nodes.

Learning Outcome

This module demonstrates non-contiguous memory structures and pointer-based relationships.

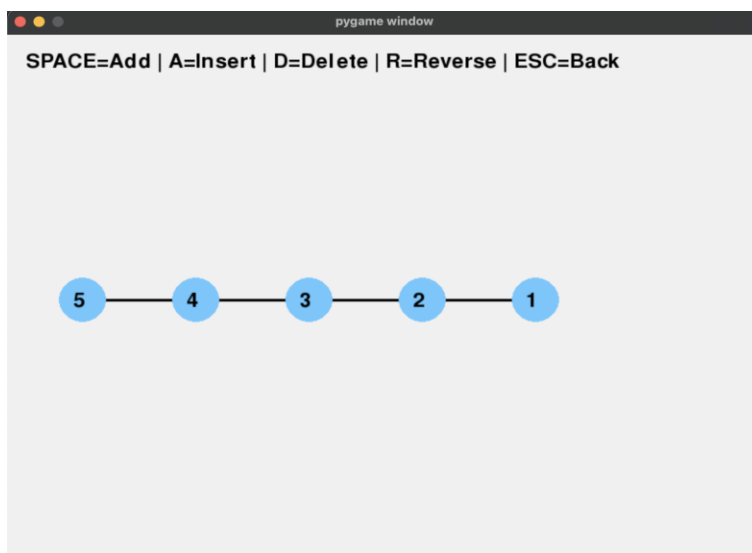


Fig 3: Linked list visualizer

2.2 Binary Search Tree (BST) Visualiser

The BST Visualiser demonstrates hierarchical node organisation and traversal behaviour.

Features

- Node insertion
- Search visualisation
- Tree traversal
- Path highlighting

Algorithms Used

- Binary Search Tree insertion
- In-order traversal
- Pre-order traversal
- Post-order traversal
- Node searching

Visualisation

Nodes are rendered in a tree hierarchy using parent-child relationships connected by edges.

Learning Outcome

The BST visualiser demonstrates recursive tree structures and efficient searching techniques.

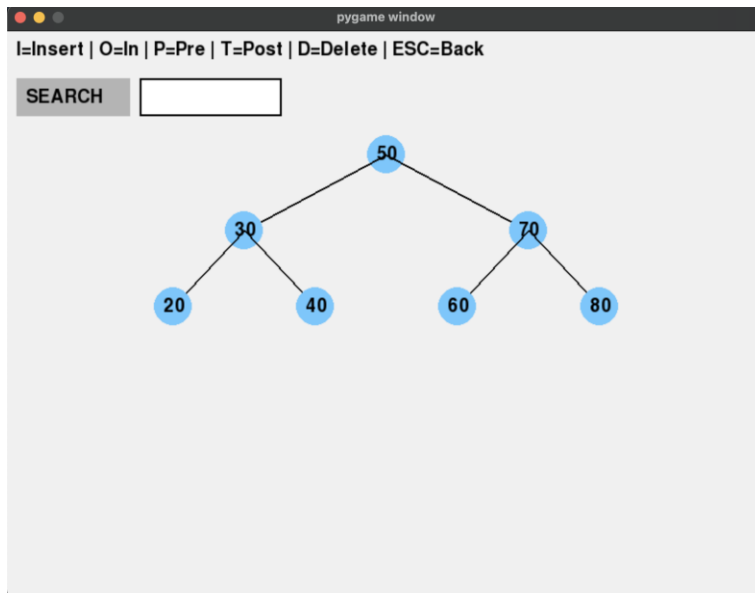


Fig 4: Binary search tree visualizer

2.3 Heap Visualiser

The Heap Visualiser demonstrates priority queue management using a Min Heap.

Features

- Dynamic event insertion
- User-controlled task input
- Priority-based ordering
- Extract minimum operation
- Heap restructuring

Algorithms Used

- Min Heap insertion
- Heapify operations
- Extract-min operation

User Interaction

- SPACE starts input mode.
- User enters priority and task name.

- ENTER confirms insertion.
- ENTER also processes the minimum-priority event.

Visualisation

Heap nodes are displayed as connected circular nodes representing parent-child relationships.

Improvements

The heap system was redesigned from random event generation to fully interactive user input. This significantly improved usability and demonstrated dynamic heap operations more clearly.

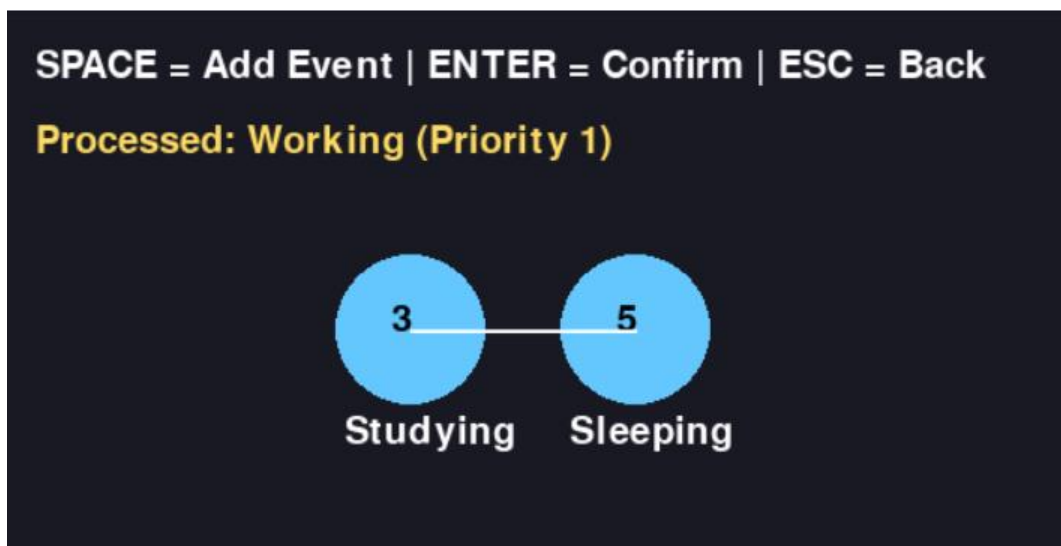


Fig 5: Heap visualizer

2.4 Graph Visualiser

The Graph Visualiser demonstrates relationships between connected nodes.

Features

- Dynamic graph generation
- Node-edge relationships
- Visual node connections

Algorithms Used

- Graph representation
- Edge connections
- Node relationships

Visualisation

Nodes are displayed as circles connected through weighted edges.

Learning Outcome

The module demonstrates adjacency relationships and graph connectivity.

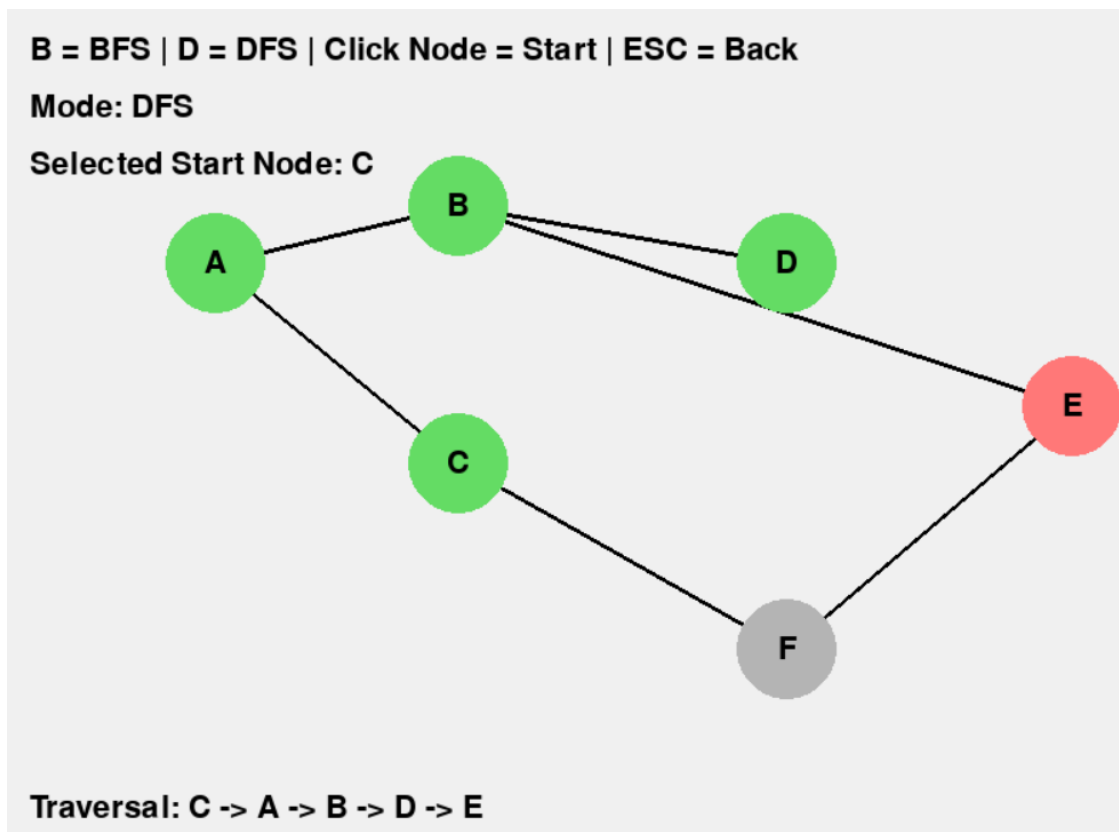


Fig 6: Graph visualizer

3. Phase 3 Implementation

3.1 Sorting Visualiser

The Sorting Visualiser demonstrates how sorting algorithms rearrange data.

Algorithms Implemented

- Bubble Sort
- Selection Sort
- Merge Sort

Features

- Animated comparisons
- Swap highlighting
- Dynamic bar rendering
- Step-by-step execution

Visualisation

Bars represent array values and dynamically move during sorting.

Learning Outcome

The visual feedback helps users understand algorithm efficiency and sorting behaviour.

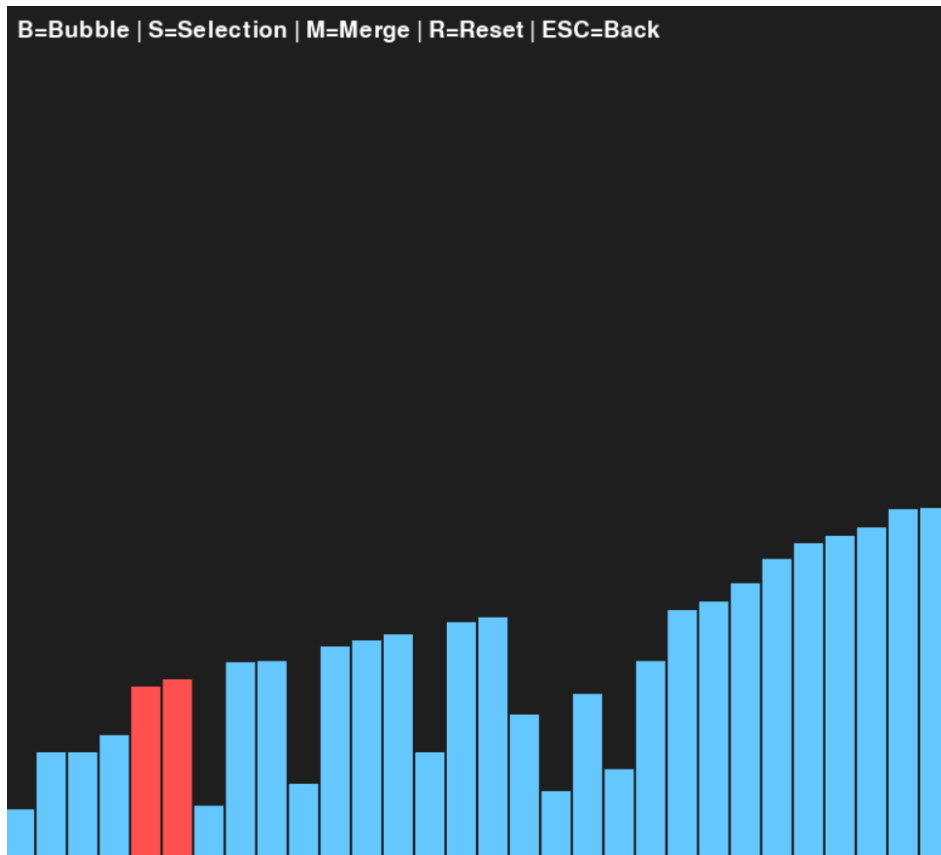


Fig 7: Sorting visualizer

3.2 Dynamic Programming (DP) Visualiser

The Dynamic Programming Visualiser demonstrates path-counting using grid-based dynamic programming.

Features

- Obstacle handling
- Dynamic path counting
- Path reconstruction
- Total path calculation
- Valid path highlighting
- Interactive controls

Algorithms Used

- Dynamic Programming Grid Traversal
- Path Reconstruction
- State Transition Computation

User Interaction

- SPACE executes the DP algorithm.
- R resets the grid.
- ESC returns to the main menu.

Visualisation

The system fills the DP grid step-by-step while visually updating valid states.

Different colours are used to represent:

Colour	Meaning
Black	Obstacle/Blocked Cell
Green	Valid Cell
Yellow	Reconstructed Valid Path
Blue	Current Processing Cell
Red	Invalid End State

Improvements

The DP module was significantly improved by adding:

- Obstacle handling
- Valid path reconstruction
- Dynamic total path calculation

- Path highlighting
- Interactive reset controls
- Improved UI spacing and readability

Learning Outcome

This module demonstrates state-based computation, recursive relationships, and optimisation concepts through visual dynamic programming.

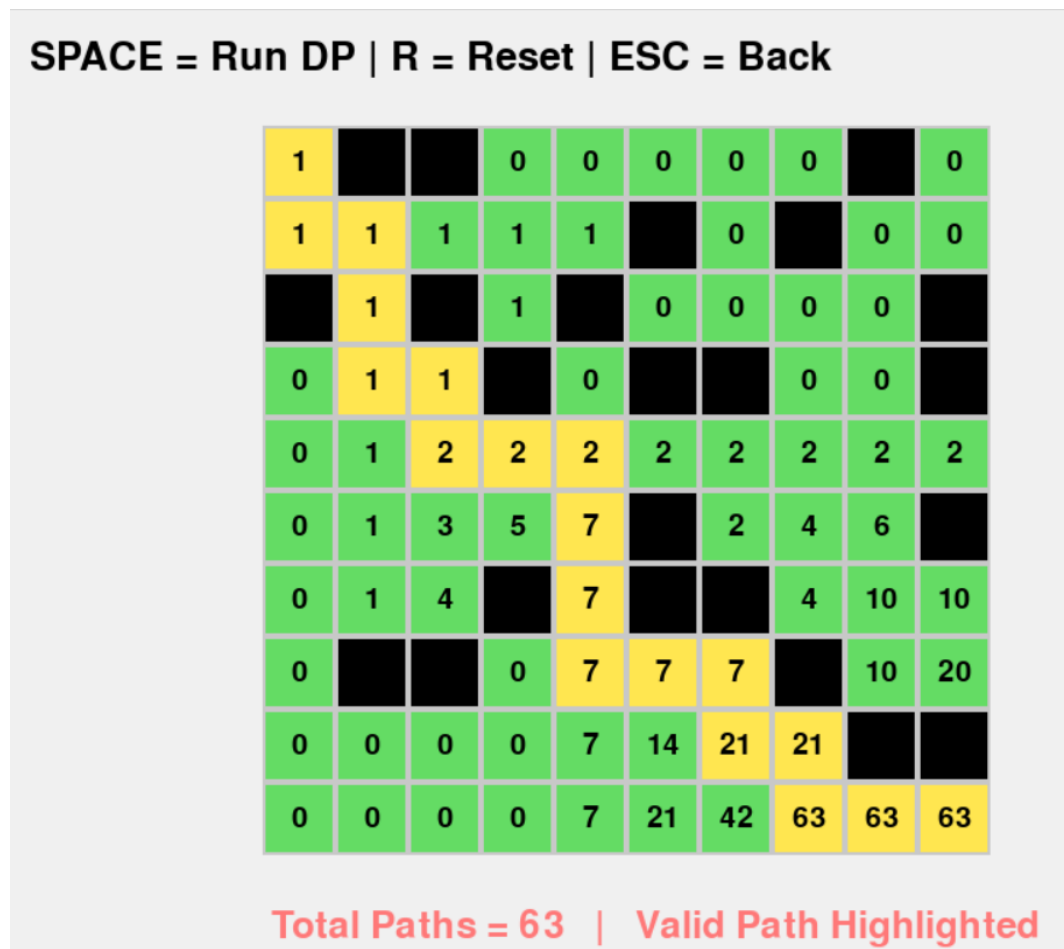


Fig 8: DP visualizer

4. Automated Testing and Benchmarking

Automated testing was implemented using Python's unittest package.

Test Files

- heap_test.py
- queue_test.py
- stack_test.py

Testing Objectives

- Validate insertion and deletion operations.
- Verify correct algorithm behaviour.
- Detect logical errors.
- Improve reliability.

Benchmarking

Basic benchmarking was performed by observing:

- Sorting execution behaviour
- Heap restructuring performance
- DP visualisation timing

Future Improvements

Additional benchmarking could include:

- Runtime comparison graphs
- Complexity measurements
- FPS performance metrics

5. Pygame Visualisation Implementation

The project uses Pygame extensively to create interactive visual feedback for each data structure and algorithm.

Visual Features

- Real-time animations
- Dynamic object rendering
- Keyboard controls
- Interactive menus
- Visual traversal updates
- Highlighting of operations

Interactivity

Each module provides keyboard controls for manipulating the structure dynamically.

Examples include:

Module	Controls
Stack	Push / Pop
Queue	Enqueue / Dequeue
Heap	Insert / Process Event
DP	Run / Reset
Sorting	Start Sorting

These interactions improve usability and support active learning.

6. Non-Functional Requirements

User Interaction

The system provides intuitive keyboard-based controls for all modules.

Integration and Navigation

All modules are integrated through a central menu system allowing seamless switching between visualisers.

Code Quality and Modularity

The project follows modular programming practices:

- Separate files for each module
- Reusable functions
- Organised structure
- Readable naming conventions
- Object-oriented principles

These design choices improved scalability and maintainability.

7. Challenges Faced

Several challenges were encountered during development.

Key Challenges

- Managing multiple visualisers in one application.
- Handling Pygame event loops correctly.
- Preventing overlapping visual elements.
- Maintaining responsive UI controls.
- Debugging heap and DP visualisation logic.
- Integrating dynamic user input into visual structures.

Solutions

The team solved these challenges through:

- Modular design
- Continuous debugging
- Step-by-step testing
- Refactoring visual layouts
- Improving interaction systems

8. Conclusion

The DSA Explorer project successfully demonstrates multiple data structures and algorithms using interactive Pygame visualisations.

The final application provides both educational value and practical implementation experience by combining:

- Data structure logic
- Algorithm visualisation
- Interactive user controls
- Modular software engineering principles

The project achieved the objectives of implementing and visualising DSA concepts across all three assignment phases.

The final system demonstrates:

- Correct algorithm implementation
- Dynamic visual feedback
- Interactive controls
- Modular architecture
- Educational usability

Overall, the project significantly improved understanding of data structures, algorithms, event-driven programming, and graphical software development using Python and Pygame.

